Computational Astrophysics Exercises

Assignments Set 1 — Summer Term 2026

Names: Karl Gauck, 6754404
Benedict Sondershaus, 6678384

Date: 27.04.26

TODO: use original exercise descriptions

Exercise 1: Floating Point Arithmetic — 3 pts

Calculate the partial sum

$$s_k = \sum_{n=1}^{k} \frac{1}{n^2} \tag{1}$$

for $k = 10^3, 10^6, 10^7, 10^8$ in both single and double precision, and analyse the convergence to the analytical limit $s_\infty = \frac{\pi^2}{6}$.

(a) Forward summation. Compute s_k for $k = 10^3, 10^6, 10^7, 10^8$. Plot $|s_k - s_\infty| / s_\infty$ and report the runtime of your program. Compare single vs. double precision.

Single precision (**f32 forwards** in the plots) is not as precise as double precision for this particular problem. The summation for single precision attains a wrong value, while the double precision calculation approaches the correct one for $k \to \infty$.

(b) Backward summation. Repeat part (a) starting from the smallest summand. Compare with (a) and explain your findings.

With backwards summation, both single and double precision approximate the correct assymptotic value correctly for $k \to \infty$. This probably stems from the fact that floating point representations are more precise the nearer the value they are representing is at 0. Thus, if the calculations start with smaller values, the higher precision in the vicinity of 0 is preserved longer.

The plots are combined for **(a)** and **(b)**:

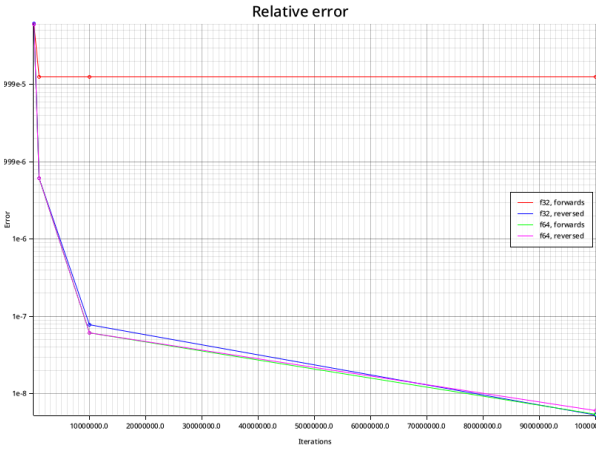


Figure 1: The relative error gets smaller for $k \to \infty$ for all methods aside from **f32 with forwards summation**. There the error stays the same for our sets of k.

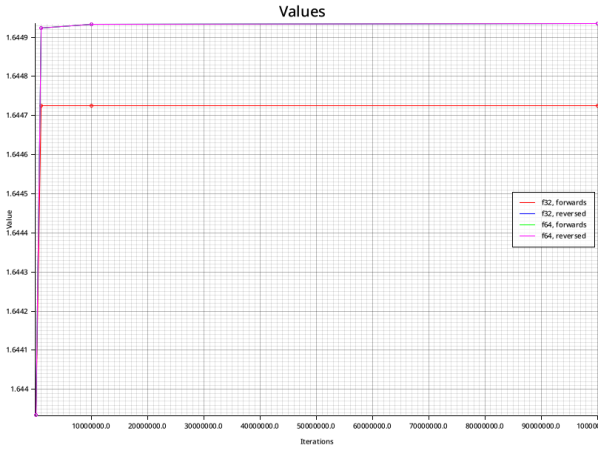


Figure 2: The error behaviour can also be seen in the real values, as **f32 forwards** computes a slightly smaller value than the other methods which are around the same value.

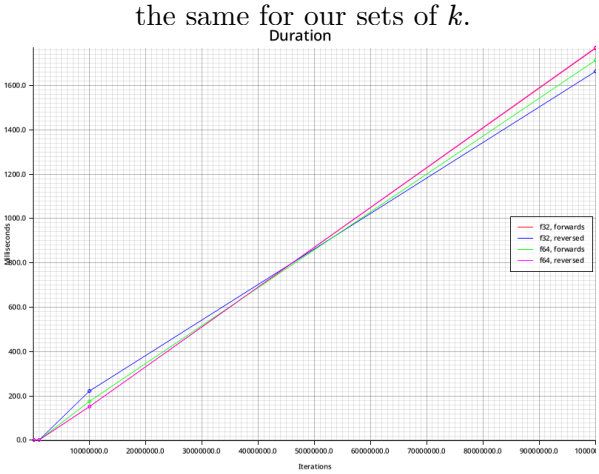


Figure 3: The duration of the summations is roughly the same for all methods, the fine deviations can't be considered as we only have sample size $n = 1$. But generally speaking, f64 reversed is the slowest and f32 reversed is the fastest.

Exercise 2: Machine Epsilon / Unit Roundoff — 2 pts

The machine epsilon ε is the smallest value satisfying $1 + \varepsilon > 1$.

Write a program that computes ε for both single and double precision. Compare your result with the values reported by `np.finfo()` (Python) or `limits.h` (C).

	Single Precision (`f32`)	Double Precision (`f64`)
measured	`0.00000005960465`	`0.0000000000000001110223024625156 8`
in `rust`	`0.00000011920929`	`0.0000000000000002220446049250313`

Our measured values are smaller than the pre-defined values of Rust. Those given epsilons are exactly double our values.

Exercise 3: Nifty Tricks — Rescaling 2 pts

Consider the computation of

$$c = \sqrt{a^2 + b^2}. \tag{2}$$

Using single precision with $a = 10^{30}$ and $b = 1$, the naive implementation returns `inf`:

```c
/* snippet.c */
float a = 1e30, b = 1.0, res = 0.0;
res = sqrt(a*a + b*b);
fprintf(stdout, "standard method res: %e\n", res);
// output: inf
```

(a) Explain why `inf` is returned for these inputs.

`inf` is returned, because when `1e30` is squared, the result would be `1e60` which is larger than $(2 - 2^{-23}) \times 2^{127} \approx 3.4028235 \times 10^{38}$, which is the largest single precision number smaller than `inf`. Thus the calculation spills into infinity.

(b) Nifty trick. Describe and implement an algebraically equivalent rescaling that avoids overflow while remaining in single precision.

The **nifty trick**™ is to simply scale down the original value, so the overflow does not happen. We used 10^{20}.

Exercise 4: Old Greek Guy Got Pie 3 pts

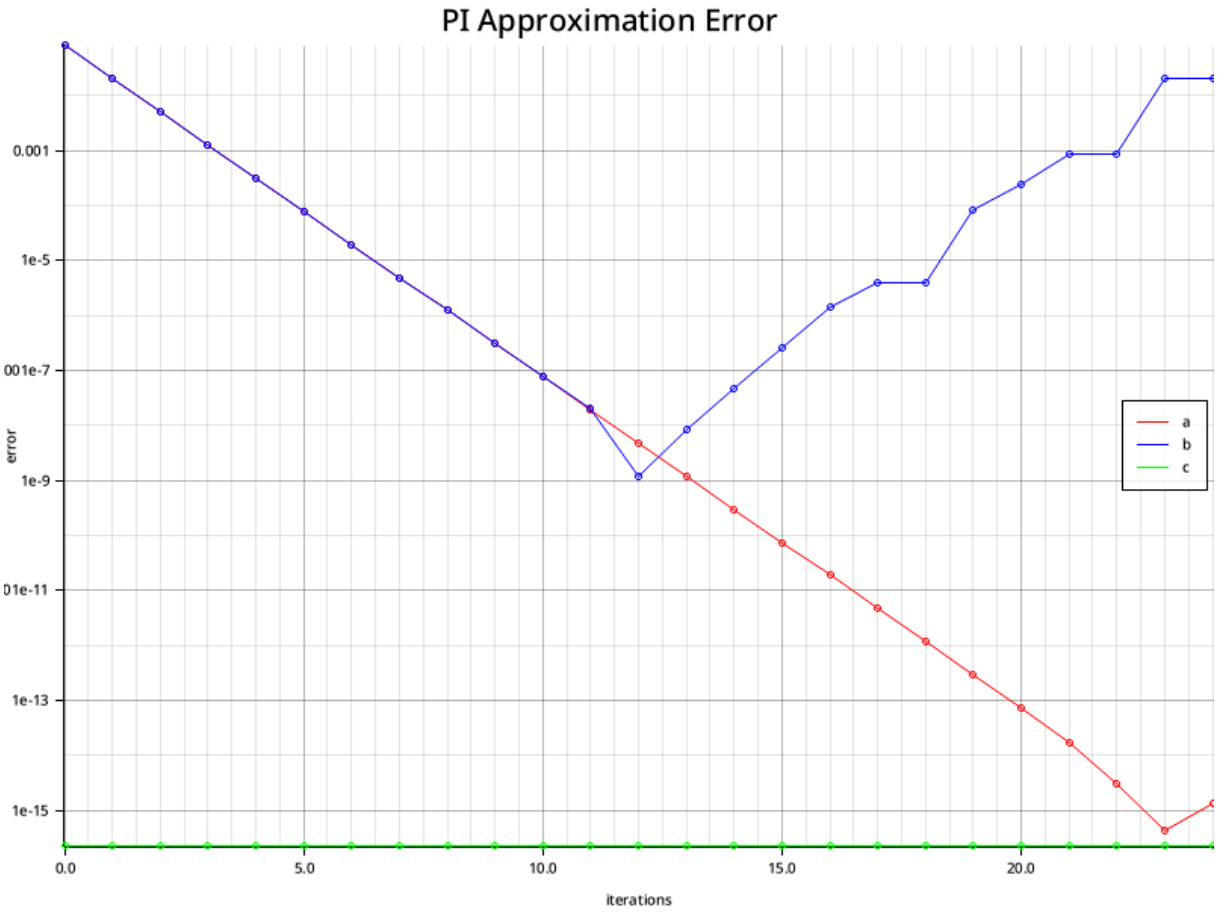
Archimedes (287–212 BC) approximated π using inscribed regular n-polygons in a unit circle of radius $\frac{1}{2}$. The perimeter of the n-polygon is

$$U_n = n \sin\left(\frac{\pi}{n}\right), \quad \lim_{n\to\infty} U_n = \pi. \tag{3}$$

Starting from the square ($A_2 = U_4 = 2\sqrt{2}$), the recursion is

$$A_{n+1} = 2^n \sqrt{2\left(1 - \sqrt{1 - \left(\frac{A_n}{2^n}\right)^2}\right)}, \tag{4}$$

(1) Implement the recursion Equation 4 and reproduce the error plot ($|A_n - \pi|$ vs. number of iterations). You should observe divergence after iteration 15.



The plot of Equation 4 is in the blue line.

(2) Explain why the error grows after iteration 15, even though double precision nominally offers $\sim 10^{-15}$ accuracy.

The error grows as the numbers being subtracted under the root get smaller and smaller and with them the error in the subtraction calculation. After a certain threshold, the error explodes and leads to the divergence.

(3) Kahan's stable reformulation replaces Equation 4 with

$$Z_n = \frac{2\left(\frac{A_n}{2^{n+1}}\right)^2}{1+\sqrt{1-\left(\frac{A_n}{2^n}\right)^2}}, \quad A_{n+1} = 2^n\sqrt{4Z_n}. \tag{5}$$

Implement this alternative and explain the improvement.

Equation 5 is shown in the graph in the red line. The improvement stems from the algorithm using a seperate variable to "save" the lost precision like a carry.